Singly Linked List – Reverse List Using Stack – Space Complexity and Algorithm

Program: –

```
#include <stack>
….
void reverseListUsingStack(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        stack<Node *> s;
        Node *current = *head;
        while (current != nullptr)
        {
            s.push(current);
            current = current->next;
        }
        *head = s.top();
        s.pop();
        current = *head;
        while (!s.empty())
        {
            current->next = s.top();
            s.pop();
            current = current->next;
        }
        current->next = nullptr;
    }
}
…..
```

```
case 15:
        {
            reverseListUsingStack(&head);


        }
```

Space Complexity Analysis

1. Input Space:

A. Parameter – Only Definition:

- ***The input to the function is a pointer to the head of the linked list (Node ** head). This pointer requires constant space, i.e., O(1).***

  This view only considers the memory for the parameters passed to the function. The single parameter `Node **head` is a pointer-to-a-pointer, which has a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

  • Single parameter head (type Node**) → one machine word.

  ⇒ Θ(1) = O(1)

B. Full – Data (Inclusive) Definition:

This more common definition includes the entire data structure. If the linked list has **n nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

• Same single word as above (Θ(1)), plus

• All nodes that can be reached from *head when the function is invoked.

If the list contains n nodes, those nodes occupy Θ(n) bytes.

⇒ Θ(n) = O(n)

2. *Auxiliary Space*:

$\boldsymbol{The\ function\ uses\ a\ stack\ (stack < Node\ *>\ s) to\ store\ the\ nodes\ of\ the\ linked\ list.}$
$\boldsymbol{The\ maximum\ number\ of\ nodes\ that\ can\ be\ stored\ in\ the\ stack\ is\ equal\ to\ the}$
$\boldsymbol{number\ of\ nodes\ in\ the\ linked\ list. Therefore, if\ the\ linked\ list\ has\ n\ nodes,}$
$\boldsymbol{the\ auxiliary\ space\ complexity\ due\ to\ the\ stack\ is\ O(n).}$

$\boldsymbol{Additionally, the\ function\ does\ not\ use\ any\ other\ data\ structures\ that\ grow}$
$\boldsymbol{with\ the\ input\ size, so\ the\ overall\ auxiliary\ space\ complexity\ is\ dominated}$
$\boldsymbol{by\ the\ stack\ usage.}$

$\boldsymbol{The\ function\ uses\ a\ few\ temporary\ pointers: current\ and\ the\ stack\ itself.}$
$\boldsymbol{The\ pointer\ current\ requires\ constant\ space, O(1), as\ it\ does\ not\ depend\ on}$
$\boldsymbol{the\ size\ of\ the\ input\ linked\ list.}$

$\boldsymbol{The\ stack, as\ mentioned\ earlier, can\ grow\ to\ hold\ all\ n\ nodes, which\ contributes}$
$\boldsymbol{O(n)\ to\ the\ space\ complexity.}$

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function, the stack is the key factor.

1. **`stack<Node *> s`**: The function declares a stack to hold pointers to the list's nodes.
2. **`Node *current`**: A single pointer is declared for traversal, which takes up constant space.
3. **Stack Population**: The first `while` loop traverses the entire linked list. For each of the **`n` nodes** it visits, it pushes a pointer onto the stack `s`.
4. **Total Space**: After the first loop, the stack will contain `n` pointers. The space required for the stack is therefore directly proportional to the number of nodes in the list. The memory usage grows linearly with the input size.

Because the stack's size depends directly on the length of the list, the auxiliary space complexity is **Θ(n)**.

Auxiliary (Extra) Space Complexity What the algorithm itself allocates in addition to its input:

• std::stack<Node > *s – The container itself is a small object on the stack (Θ(1)). – As each of the n node addresses is pushed, the underlying container (deque or vector) allocates space to hold n pointers. – Memory consumed for these pointers = n × sizeof(Node)* → Θ(n).

• Local variables – Node *current and a few compiler temporaries → Θ(1).

• No recursion, no other dynamic structures.

⇒ Total auxiliary space in the worst case: Θ(n) = O(n)

Algorithm

$\boldsymbol{ReverseListUsingStack(START)}$

1. $[START\ is\ NULLPTR?\ ]if\ START\ =\ NULL, then:$

$Write:``\ Error: List\ is\ Empty.``$

$Return\ and\ exit.$

$ELse$

$[Create\ an\ empty\ stack\ ]STACK$

$SET\ CURRPTR := START\begin{bmatrix} CURRPTR\ will\ now\ point\ to\ FIRST\ NODE \\ i.e.\ by\ holding\ value\ obtained\ by\ START\ or\ Head\ pointer \end{bmatrix}$

$[LOOP\ will\ help\ to\ Traverse\ from\ FIRST\ NODE\ TO\ LAST\ with\ help\ of\ CURRPTR]$

$Repeat\ while\ CURRPTR\ \neq NULL, then:$

$PUSH(STACK\ , CURRPTR)[PUSH\ ADDRESS\ of\ NODE\ that\ CURRPTR\ will\ point\ to\ while\ traversing]$

$- -$

$Translating\ \ PUSH(STACK\ , CURRPTR)$

1. $[Stack\ already\ filled?\ ]If\ TOP = MAXSTK, then:$

$Write:``OVERFLOW``$

$Return\ and\ Exit.$

2. $SET\ TOP := TOP + 1\ [INCREASES\ TOP\ by\ 1]$

3. $SET\ STACK[TOP] := CURRPTR\ \begin{bmatrix} Inserts\ Addresses\ of\ Nodes\ pointed\ by\ CURRPTR \\ in\ \ TOP\ position\ of\ STACK. \end{bmatrix}$

4. $Return.$

$- -$

$SET\ CURRPTR := LINK[CURRPTR][CURRPTR\ will\ now\ point\ to\ NEXT\ NODE]$

$[END\ of\ Loop]$

$SET\ START := STACK[TOP]$ $[Head\ or\ START\ now\ point\ to\ last\ NODE]$

$POP(STACK)[No\ need\ of\ TOP\ of\ the\ STACK\ which\ holds\ the\ Address\ of\ last\ Node]$

– –

$Translating\ \ POP(STACK\ )$

$1.[STACK\ has\ an\ item\ to\ be\ \ removed?\ ]If\ TOP = 0, then:$

$Write: ``UNDERFLOW``$

$Return\ and\ Exit.$

$2.SET\ TOP := TOP - 1\ [DECREASE\ TOP\ by\ 1.\ ]$

$3.Return.$

– –

$SET\ CURRPTR\ := START\ \left[\begin{array}{c}Now\ CURRPTR\ will\ point\ to\ LAST\ NODE's\ address, as\ START\ point\ to \\ LAST\ NODE.\end{array}\right]$

$[LOOP\ will\ help\ to\ Traverse\ from\ LAST\ NODE\ TO\ FIRST\ with\ help\ of\ CURRPTR]$

$Repeat\ while\ STACK[TOP] \neq -1\ i.e.STACK\ is\ not\ EMPTY\ , then:$

$SET\quad LINK[CURRPTR] := STACK[TOP]\left[\begin{array}{c}LINK\ of\ NODE\ that\ CURRPTR\ points\ to\ gets \\ assigned\ to\ the\ address\ hold\ by\ TOP\ of\ the \\ STACK.\end{array}\right]$

$POP(STACK)$

– –

$Translating\ \ POP(STACK\ )$

$1.[STACK\ has\ an\ item\ to\ be\ \ removed?\ ]If\ TOP = 0, then:$

$Write: ``UNDERFLOW``$

$Return\ and\ Exit.$

$2.SET\ TOP := TOP - 1\ [DECREASE\ TOP\ by\ 1.\ ]$

$3.Return.$

– –

$SET\ CURRPTR := LINK[CURRPTR]\left[\begin{array}{c}This\ helps\ CURRPTR\ to\ point\ 2nd\ LAST\ NODE\ to \\ FIRST\ NODE\ by\ getting\ assigned\ to\ LINK\ of\ NODE \\ that\ CURRPTR\ points\ to.\end{array}\right]$

$[END\ of\ Loop]$

$SET\ LINK[CURRPTR] := NULL\ [MAKING\ LINK\ of\ FIRST\ NODE\ to\ NULL\ i.e.MAKING\ FIRST\ NODE\ to\ LAST\ NODE]$

$[END\ of\ If\ structure]$

$[END\ OF\ ALGORITHM]$

$Note: In\ C++\ using\ \ include\ <stack>, the\ pop()\ \ function\ only\ removes\ the\ top\ element\ from\ the\ stack;$ $it\ does\ not\ return\ it.$

$i.e.\ in\ General\ case\ , it\ should\ have\ been:$

$Translating\ \ POP(STACK\ , CURRPTR)$

$1.\ [STACK\ has\ an\ item\ to\ be\ \ removed?\ ]If\ TOP = 0, then:$

$Write: ``UNDERFLOW``$

$Return\ and\ Exit.$

$2.\ SET\ CURRPTR := STACK[TOP]\ [i.e.\ first\ return\ Address\ stored\ in\ TOP\ of\ the\ STACK\ to\ CURRPTR]$

$2.\ SET\ TOP := TOP - 1\ [DECREASE\ TOP\ by\ 1.]$

$3.\ Return.$

$- -$

$But\ as\ we\ are\ using\ pop()\ function\ of\ \#include < STACK > it\ is:$

$- -$

$Translating\ \ POP(STACK\ )$

$1.\ [STACK\ has\ an\ item\ to\ be\ \ removed?\ ]If\ TOP = 0, then:$

$Write: ``UNDERFLOW``$

$Return\ and\ Exit.$

$2.\ SET\ TOP := TOP - 1\ [DECREASE\ TOP\ by\ 1.]$

$3.\ Return.$

$- -$

$i.e.\ JUST\ removing\ the\ top\ node\ .\ The\ pop()\ function\ in\ C++\ simply\ removes\ the\ top\ element\ without\ returning\ it,$ $which\ is\ different\ from\ a\ custom\ stack\ implementation\ where\ we\ might\ want\ to\ return\ the\ value\ of\ the$ $removed\ element.$